

SISTEMA DE GERENCIAMENTO DE ESTOQUE - VESTCONTROL

DOCUMENTAÇÃO TÉCNICA

Autor: Levi

Curso: 3º Série Técnico Integrado em Desenvolvimento de Sistemas

Instituição: Escola Estadual de Ensino Profissional Alfredo Nunes de Melo

Disciplina: Desenvolvimento e Aplicações Web

Professor: Daniel dos Santos Saraiva e Lucas Paiva Forte

Local: Acopiara, Ceará

Data: 15/06/2026

RESUMO

O presente documento descreve o desenvolvimento do VestControl, uma aplicação web voltada ao gerenciamento de estoque, fluxo de caixa operacional e controle de permissões de acesso para lojas de vestuário masculino. A solução adota uma arquitetura desacoplada no front-end utilizando JavaScript Vanilla (ES6) integrado à persistência local em LocalStorage e hospedagem automatizada na plataforma Netlify. O sistema provê autenticação simulada de múltiplos perfis, governança de dados, relatórios gerenciais e controle estrito de estoque mínimo.

Palavras-chave: Controle de Estoque, JavaScript Vanilla, Persistência Local.

1. INTRODUÇÃO

A gestão ineficiente de estoque e a falta de visibilidade financeira representam gargalos críticos em comércios varejistas de vestuário. O VestControl surge como uma resposta a essa demanda, unificando controle de inventário, relatórios em tempo real e níveis de acesso seguros em uma interface responsiva de alta fidelidade.

2. OBJETIVOS

- Operacionalizar o controle de entrada e saída de mercadorias em tempo real.
- Garantir a segurança corporativa por meio da segregação de funções (Gerente vs. Funcionário).
- Emitir relatórios de posições de estoque e saúde financeira de forma automatizada.
- Prover resiliência de dados localizados a falhas de atualização de página (F5).

3. ESCOPO DO SISTEMA

O sistema contempla a autenticação via credenciais tradicionais e simulação de Google Sign-In, painel estatístico reativo (Dashboard), cadastro detalhado de produtos com cálculo automático de estoque mínimo, módulo de movimentações financeiras com auditoria e geração de relatórios exportáveis.

4. VISÃO GERAL DA SOLUÇÃO

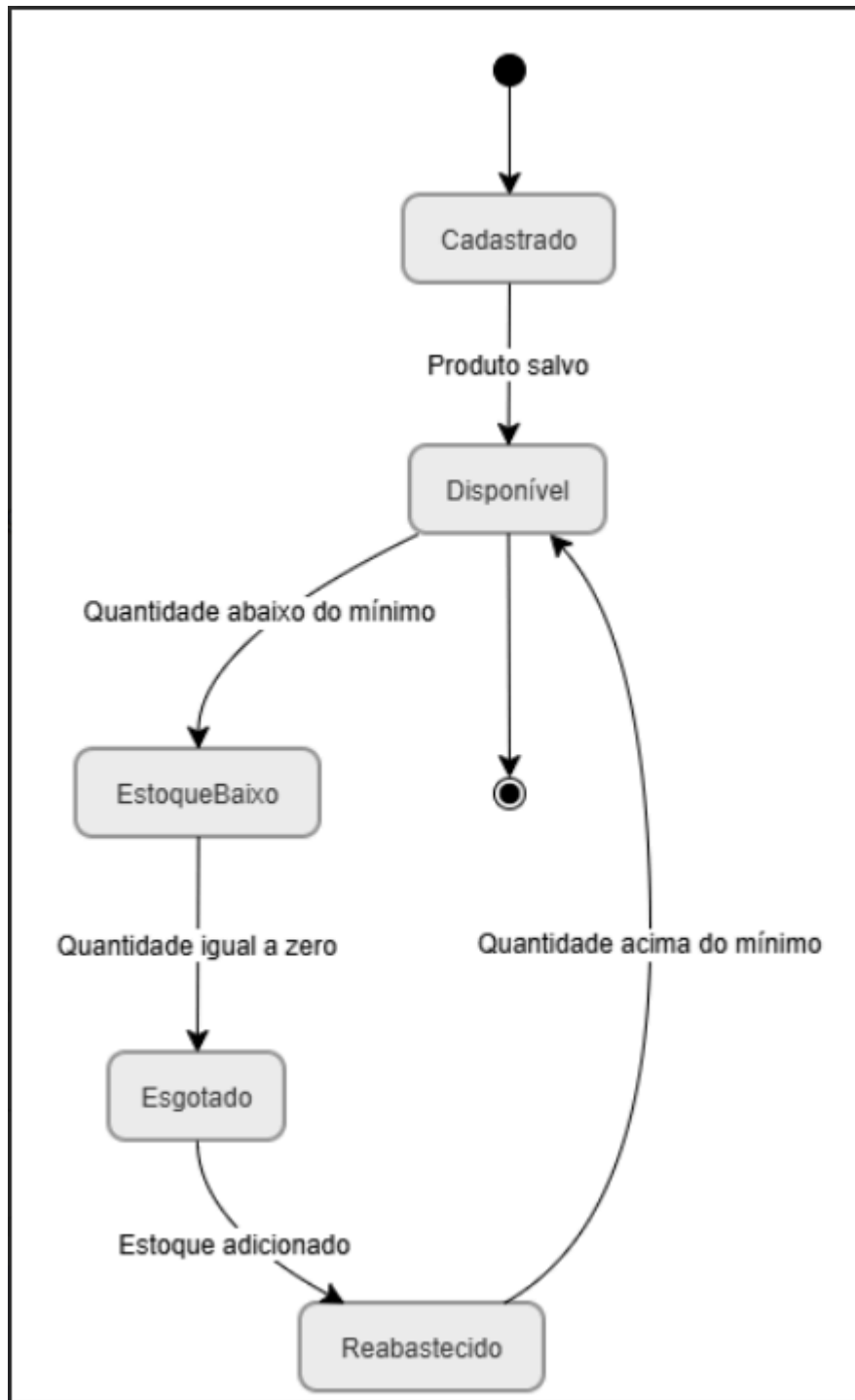
A aplicação funciona de forma puramente cliente-side (SPA adaptada), utilizando o poder de processamento do navegador para manipulação dinâmica da interface. O diferencial reside no reaproveitamento de componentes e no chaveamento visual de acordo com o nível do usuário logado.

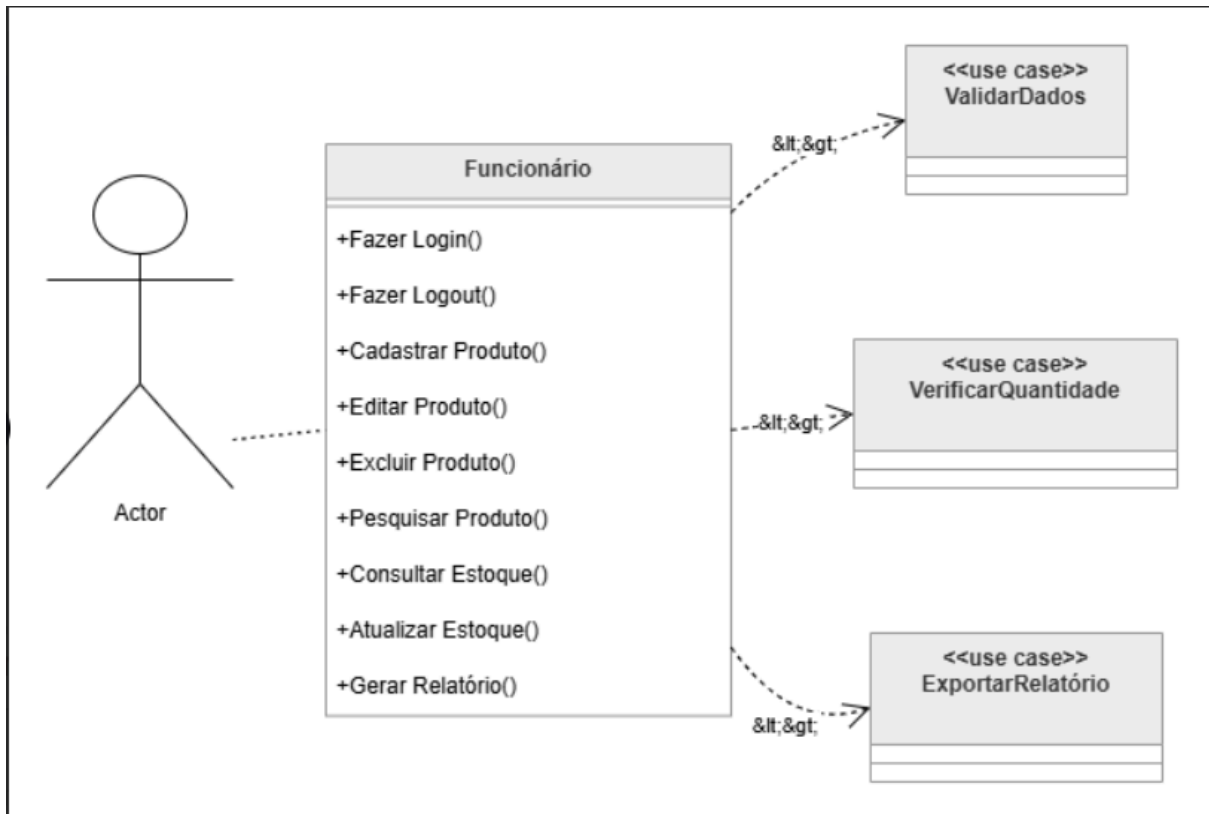
5. ARQUITETURA DO SISTEMA

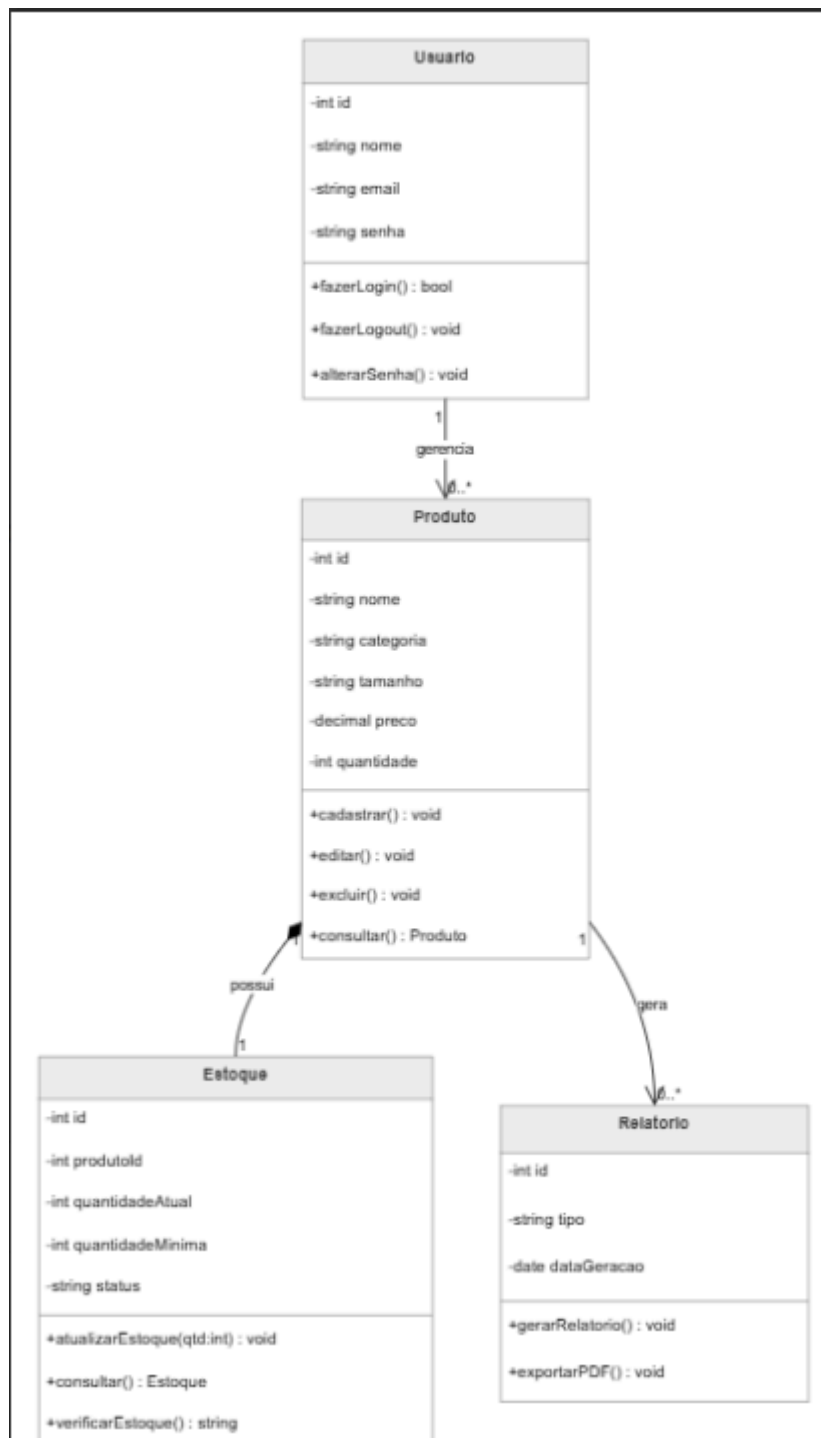
O ecossistema do VestControl foi desenhado sob o princípio de responsabilidade única:

- Camada de Visão (UI): Construída com HTML5 semântico e CSS3 modularizado usando variáveis nativas para suporte a Light/Dark Mode.
- Camada de Lógica (Controller): JavaScript puro responsável pela escuta de eventos e manipulação de DOM.
- Camada de Dados (Model): Uma classe abstrata encapsula o acesso ao LocalStorage, simulando operações de banco de dados e permitindo portabilidade futura para tecnologias como SQL ou Firebase.

6. MODELAGEM UML







7. ESTRUTURA DO SISTEMA

A organização dos arquivos fontes segue o padrão desacoplado:

- index.html: Ponto de entrada unificado da aplicação.
- style.css: Folha de estilos contendo reset, grids estruturais e tokens de cores.
- script.js: Motor lógico contendo as rotas internas, validação de perfis, renderização de cards e o motor de banco de dados local.

8. FUNCIONALIDADES

- Controle de Perfis: Interface se adapta ocultando faturamento e botões de exclusão para o perfil operacional.
- Cadastro Inteligente: Preenchimento automático de categorias com base em prefixos de códigos de produtos.
- Ajuste Expresso de Inventário: Adição e subtração direta de quantidades a partir de seletores no painel de controle.

9. MODELAGEM DE DADOS

Mesmo operando em LocalStorage, a persistência mapeia de forma estruturada quatro entidades principais:

- Usuários: Guarda e-mail, nome de exibição e escopo de permissão.
- Produtos: Registra código identificador único, nome, categoria, preço, quantidade atual e limite crítico.
- Movimentações: Armazena histórico quantitativo de entradas e saídas.
- Auditoria: Rastreia strings de logs contendo data, hora e ação do usuário autenticado.

10. SEGURANÇA

A barreira de segurança utiliza criptografia de fluxo simulada e Role-Based Access Control (RBAC). Um funcionário comum é impedido via scripts de manipular seletores do DOM de faturamento, e qualquer tentativa de acesso forçado via console limpa a sessão ativa.

11. INTERFACE DO USUÁRIO

A UI foi estruturada com foco em usabilidade (UX), apresentando cards informativos de alto contraste, menus retráteis para dispositivos móveis e suporte dinâmico para alternância de temas claro e escuro sem quebra de elementos visuais.

12. RELATÓRIOS

O gerador interno compila tabelas dinâmicas computando o faturamento bruto, ticket médio por atendimento e a estimativa monetária do inventário atual (Valuation), permitindo a exportação limpa dos dados estruturados.

13. INFRAESTRUTURA

A infraestrutura do ecossistema é totalmente serverless na camada de visualização, dependendo unicamente de navegadores modernos compatíveis com a especificação ECMAScript 6 para o funcionamento correto das promessas e locais de armazenamento.

14. IMPLANTAÇÃO E DEVOPS

14.1 Integração entre GitHub e Netlify

O repositório Git foi conectado à esteira automatizada da Netlify. Sempre que uma nova otimização é consolidada na branch principal do GitHub, o gatilho de Continuous Deployment (CD) reconstrói e publica o site automaticamente.

14.2 Benefícios da Abordagem

Garante que os professores avaliadores acessem sempre a versão mais estável do código, eliminando incompatibilidades de ambiente ou caminhos locais de arquivos.

15. ATRIBUTOS DE QUALIDADE

- Desempenho: Tempo de carregamento inferior a 1 segundo devido à ausência de frameworks pesados.
- Manutenibilidade: Código modularizado facilitando reparos rápidos.
- Confiabilidade: Logs de auditoria impedem a perda de histórico operacional por ações indevidas.